

Cryptography: asymmetric encryption and applications

Cataldo Basile

< cataldo.basile @ polito.it >

Politecnico di Torino

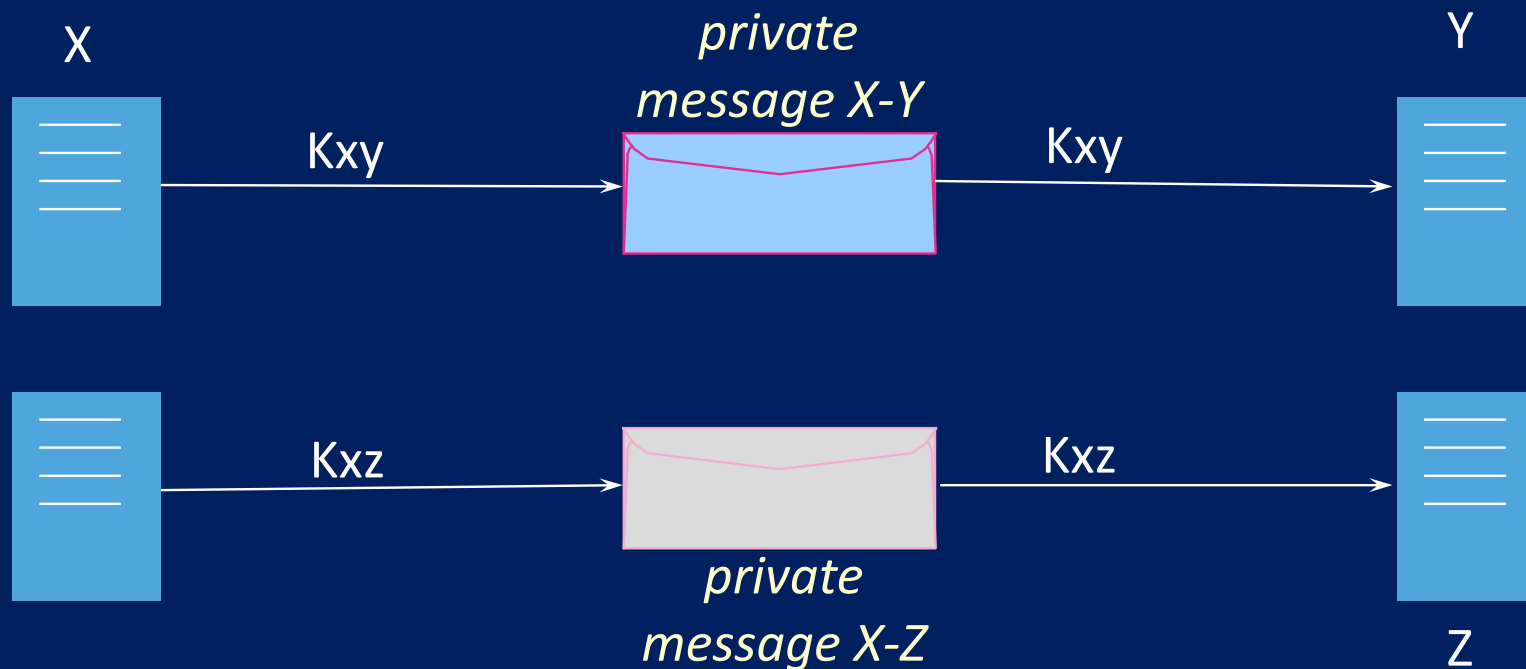
Dip. Automatica e Informatica

Agenda

- symmetric crypto: limitations
- asymmetric cryptography
 - general framework
 - RSA
- applications
 - key agreement
 - key derivation
 - symmetric authentication
- ECC
- digital certificates
- electronic documents

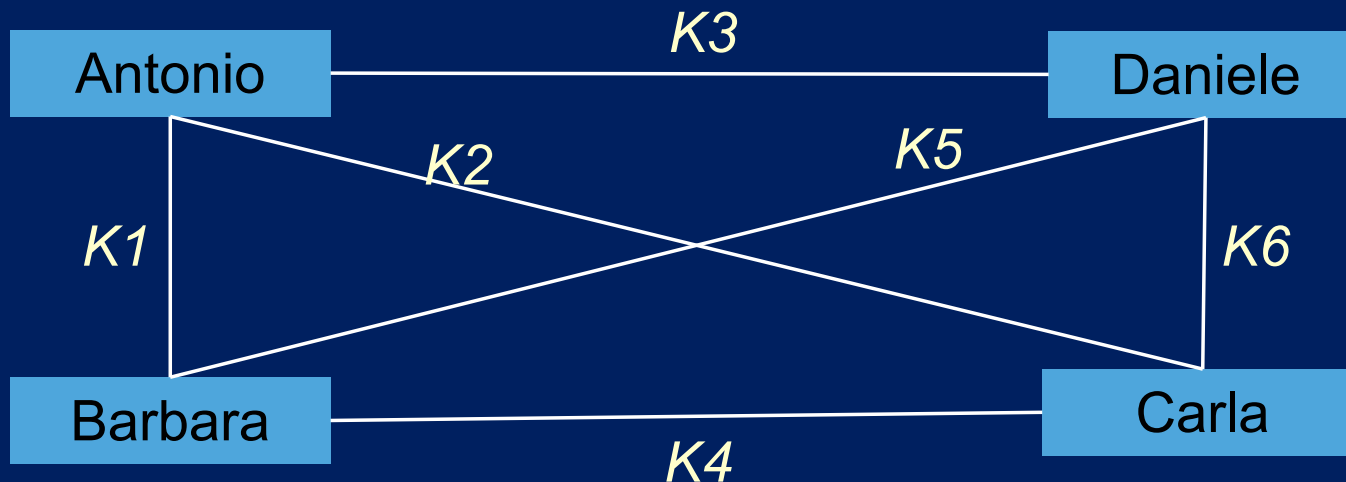
Symmetric crypto: key management

- one secret key for both operations
- one key needed for each pair of users (or group)



Key distributions

- when N persons need to secretly communicate, $N \times (N-1) / 2$ keys need to be agreed upon:
 - OOB (Out-Of-Band) distribution
 - ad hoc key agreement/key distributions methods



Public key cryptography

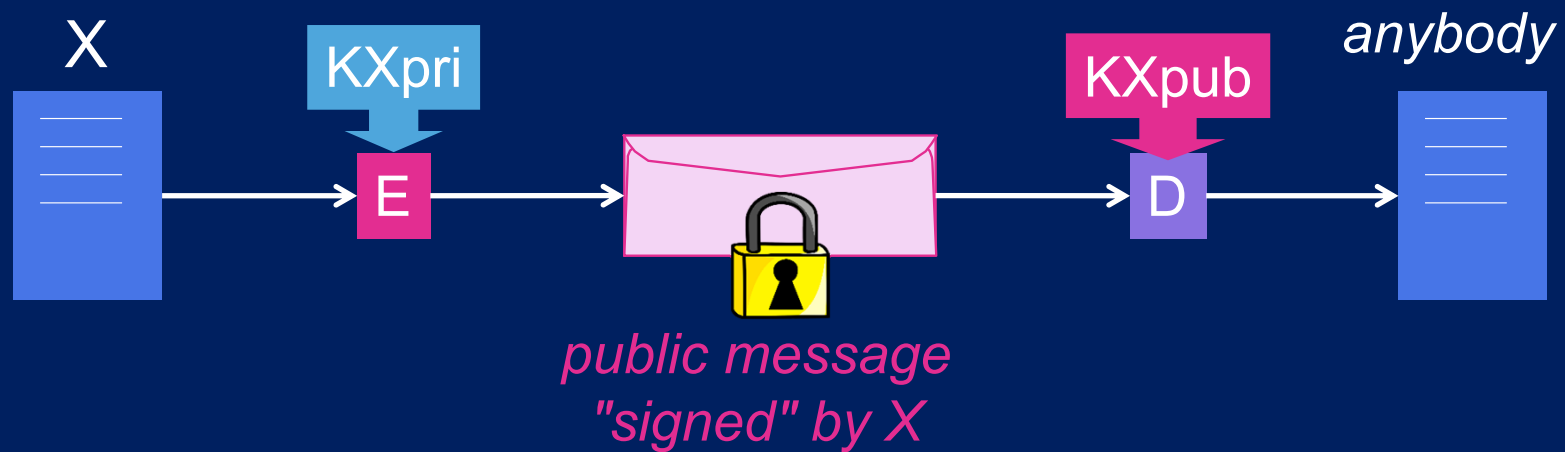
- asymmetric algorithms (aka **public key algorithms**)
 - two distinct keys
 - for the two operations encrypt/decrypt
 - encryption/decryption algorithms may also be different
 - but they may also be very different
- **computationally heavy**
 - cannot be used to encrypt large quantity of data
 - good for distributing a few bytes
 - e.g., secret keys
 - good for digital signatures
 - if combined with hash functions
- **main (classical) algorithms are**
 - Diffie-Hellman, RSA, DSA, El Gamal, ECDSA...

Asymmetric keys

- keys are generated in pairs
 - they must have inverse functionalities
 - encryption operations made with one key, must be annihilated when the other key (of the pair) is used
 - cannot be simply random
 - must share some mathematical properties
- semantics of asymmetric keys
 - one key is kept secret
 - hence, it becomes the private key
 - one is distributed to everyone
 - hence becomes the public key
 - no confidentiality issues must affect this second one
 - in some cryptosystems, public and private keys are indistinguishable
 - e.g., RSA
 - public vs. private is just a matter of choice
 - in some cryptosystems, they are different

Data authentication

- asymmetric encryption with the private key
 - e.g., the author of a document
 - ensures data authentication and integrity
 - both or none
 - usually, only encrypt the digests → the digital signature



Confidentiality without shared secrets

- generate a **secret message** for a particular receiver given only its public key



Public key algorithms

- **RSA (Rivest - Shamir - Adleman)**
 - product of prime numbers
 - inverse: factoring of the result
 - finite fields (modulo n)
 - secrecy and digital signature
- **DSA (Digital Signature Algorithm)**
 - compute the power
 - inverse: logarithm of the result
 - only for digital signatures
 - for encryption, combine it with El-Gamal

RSA – the algorithm (I)

- generation
 - public module $n = p q$ known to anybody
 - p and q are prime, large, and secret
 - $PHI = (p-1)(q-1)$
 - choose a number (public exponent) e such that arbitrarily $1 < e < PHI$ and it is relatively prime with respect to PHI
 - compute the (private exponent) $d = e^{-1} \bmod PHI$
 - if an attacker comes to know p and q , then he can compute d
 - you can also compute d from n and e
- public key = (n, e)
- private key = (n, d)

RSA – the algorithm (II)

- RSA may cipher/decipher only data whose value is less than the value of the module n
 - plaintext: $P < n$
 - encrypt: $C = P^e \bmod n$
 - decrypt: $P = C^d \bmod n$
- the roles of e and d are interchangeable because $(P^d)^e \bmod N = (P^e)^d \bmod n$
 - the complexity of the operations depends upon the number of bits with value 1 in the exponents e and d

Hard cryptographic problems

- RSA PROBLEM

- given (n,e) public $\rightarrow$ compute d

- factorization PROBLEM

- given (n,e) compute first p e q s.t. $n=pq$
- having factored n , compute d

- factorization is the hardest problem

- even if you know e and d you cannot factor n

Public keys: lenghts

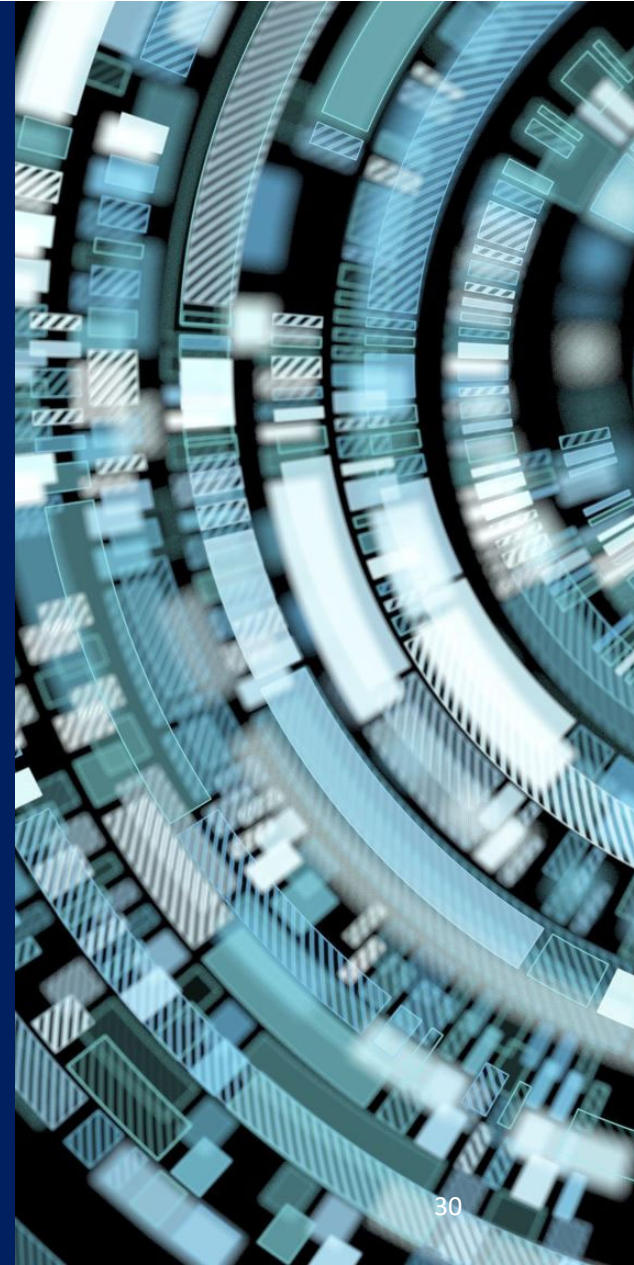
- estimation of attack times (without quantum computers)
 - 512 bits can be attacked in weeks
 - 1024 bits can be attacked in months
 - 2048 bits offer an appropriate security level for several years

<https://web.archive.org/web/20130507091636/http://www.rsa.com/rsalabs/node.asp?id=2092>

<https://wiki.mozilla.org/CA:MD5and1024>

Applications of the asymmetric crypto

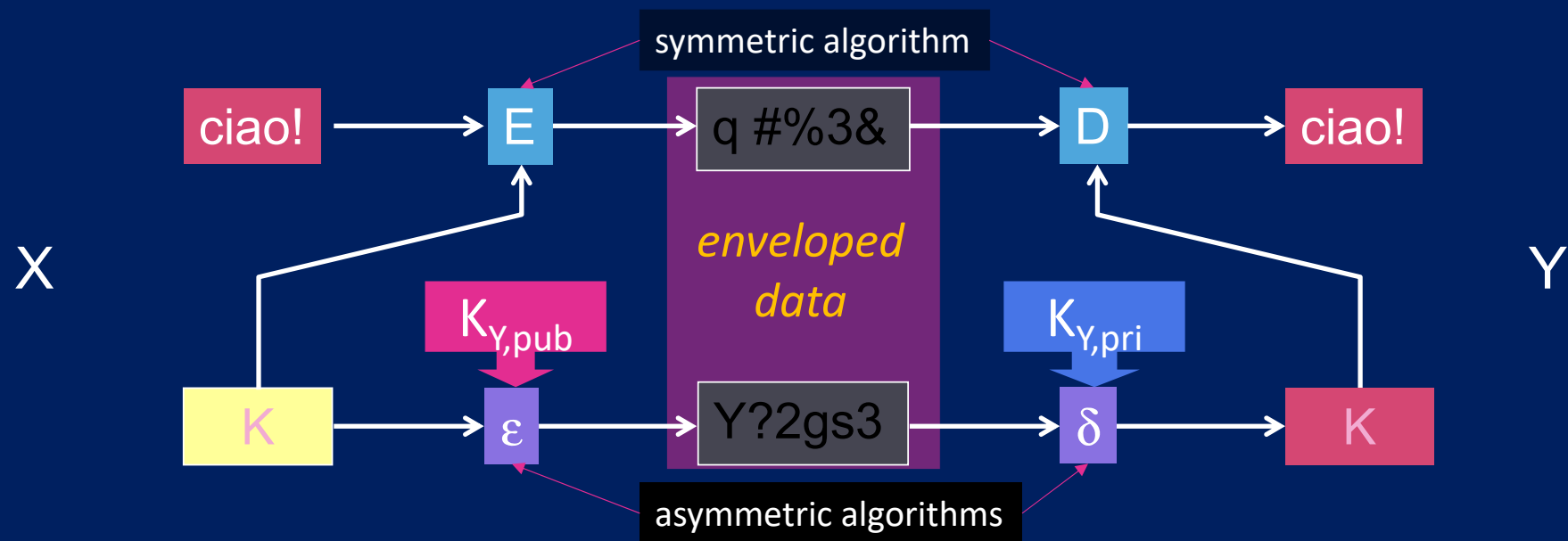
13/04/2026 - Cryptography II - © C. Basile (2026)



Key agreement / key exchange

1. Key distribution and encryption with asymmetric cryptography

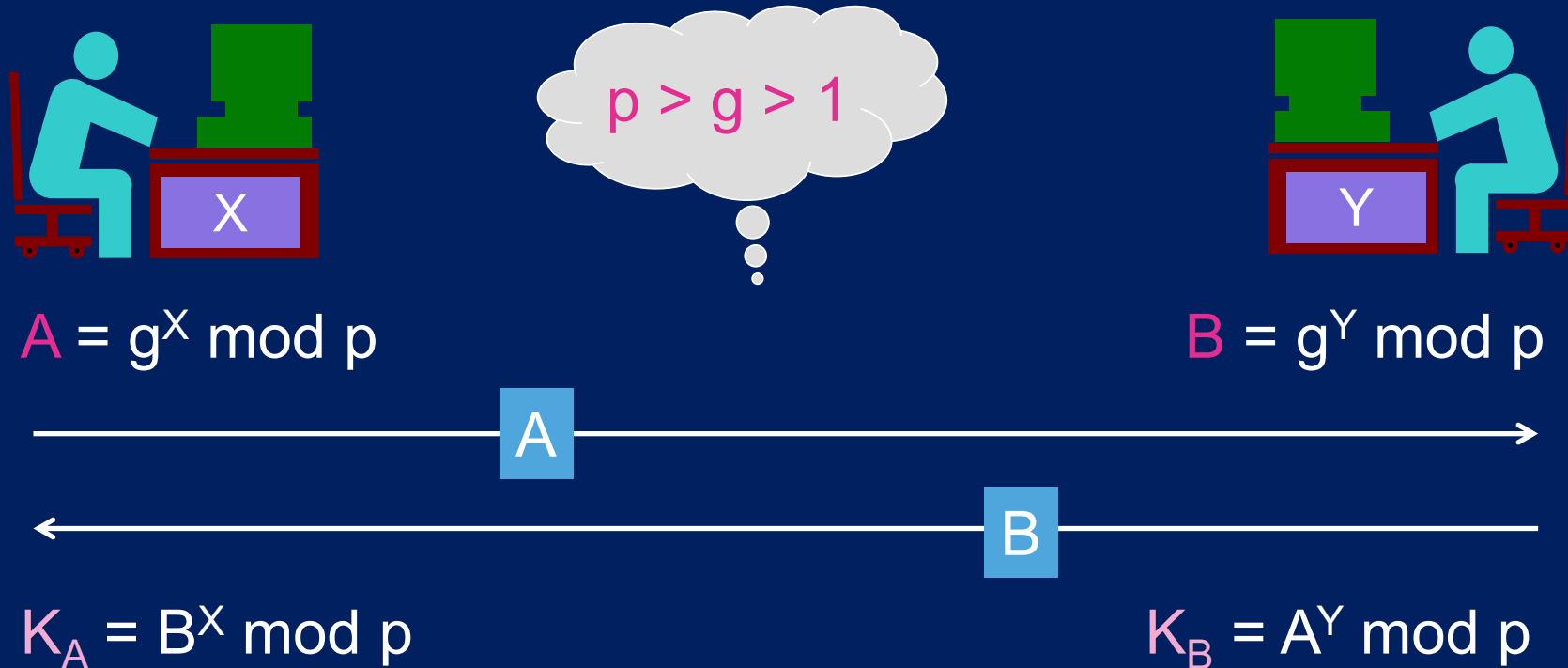
- randomly generate a message key K encrypt the message with symmetric algorithms using K
- encrypt the key K with asymmetric algorithms (e.g., RSA)



Diffie-Hellman

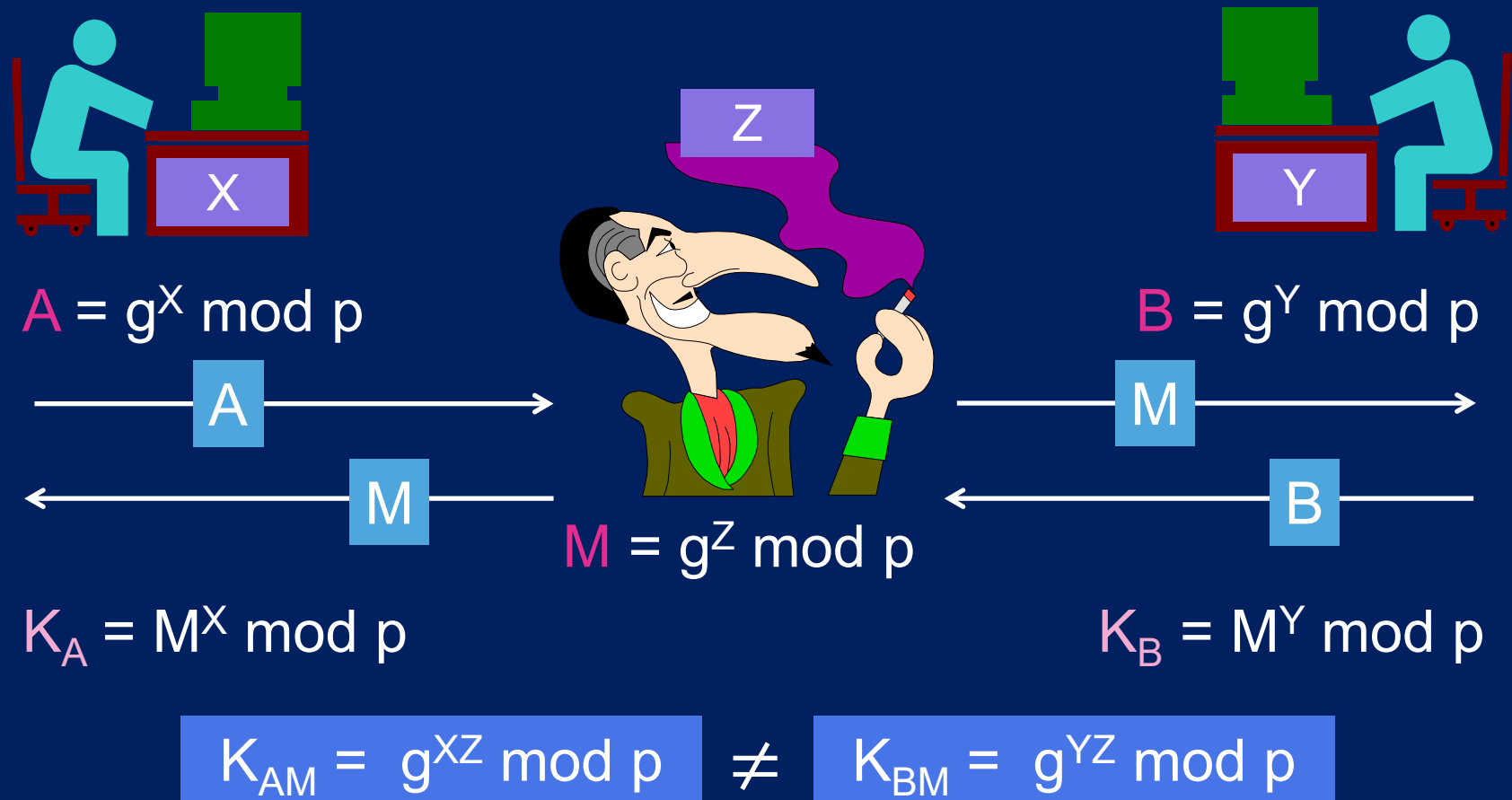
- the first public-key algorithm invented
- used to agree on a secret key
 - key agreement
- resist the sniffing attack
- if the attacker can manipulate data, then it is possible a *Man-in-the-Middle attack*
 - i.e., requires pre-authentication to work in the wild

Diffie-Hellman (DH)



$$K_A = K_B = g^{XY} \text{ mod } p = K_{AB}$$

DH: man-in-the-middle attack

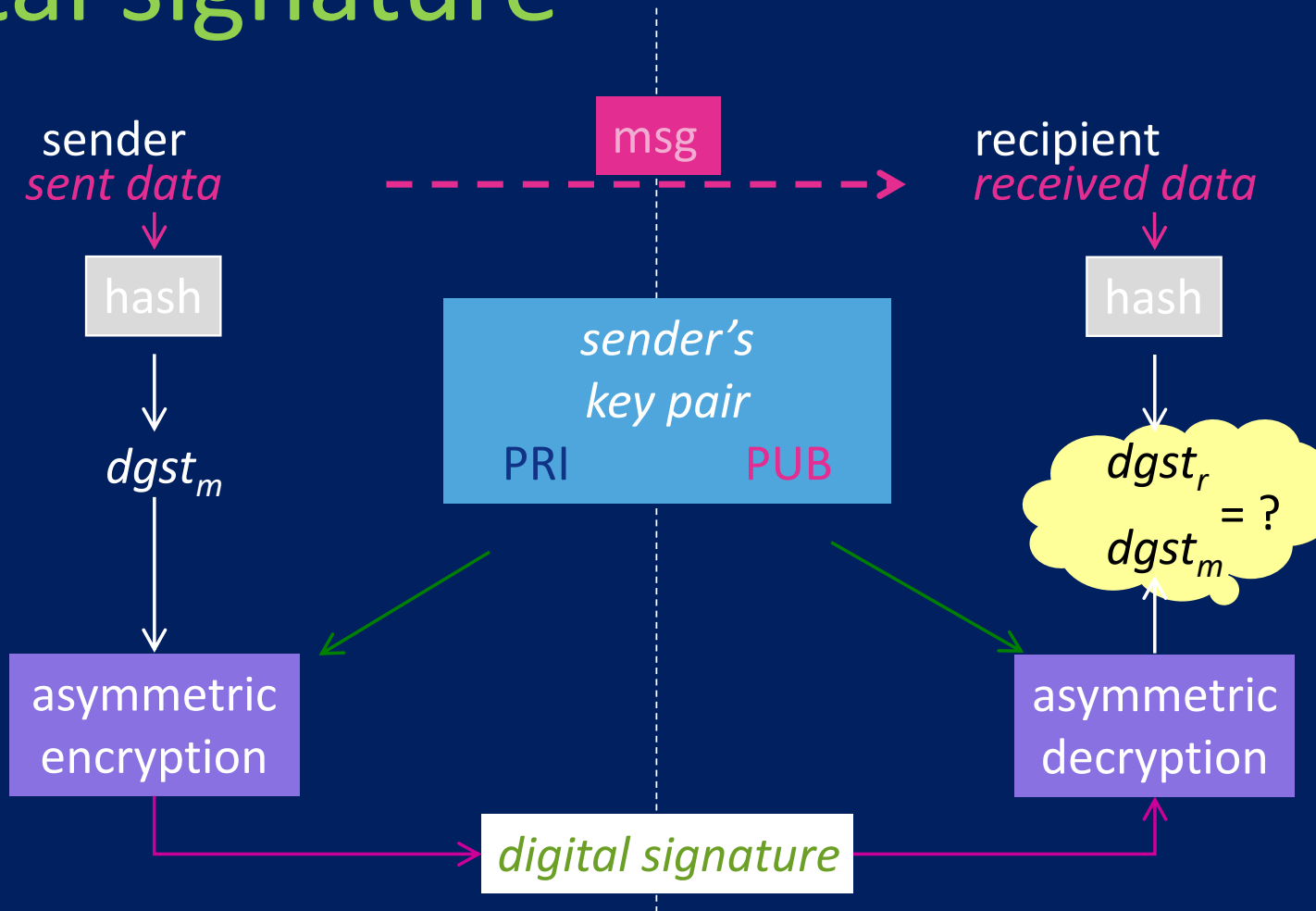


Autenticazione

3. Data authentication with asymmetric crypto

- compute a digest of the data to authenticate, encrypted with the private key of the originator
 - (signer S) $H = \text{enc}(S_K_{\text{pri}}, \text{hash}(M))$
 - (send) $M || H$
 - (verifier V) $X = \text{dec}(S_K_{\text{pub}}, H)$
 - (verification) *if ($X == \text{hash}(M)$) then OK else ALARM!*
- who knows the public key of the originator can extract the digest and compare it with the one computed on the received data
- **DIGITAL SIGNATURE** (the «real» one)
 - also need to manage padding to avoid attacks

Digital signature



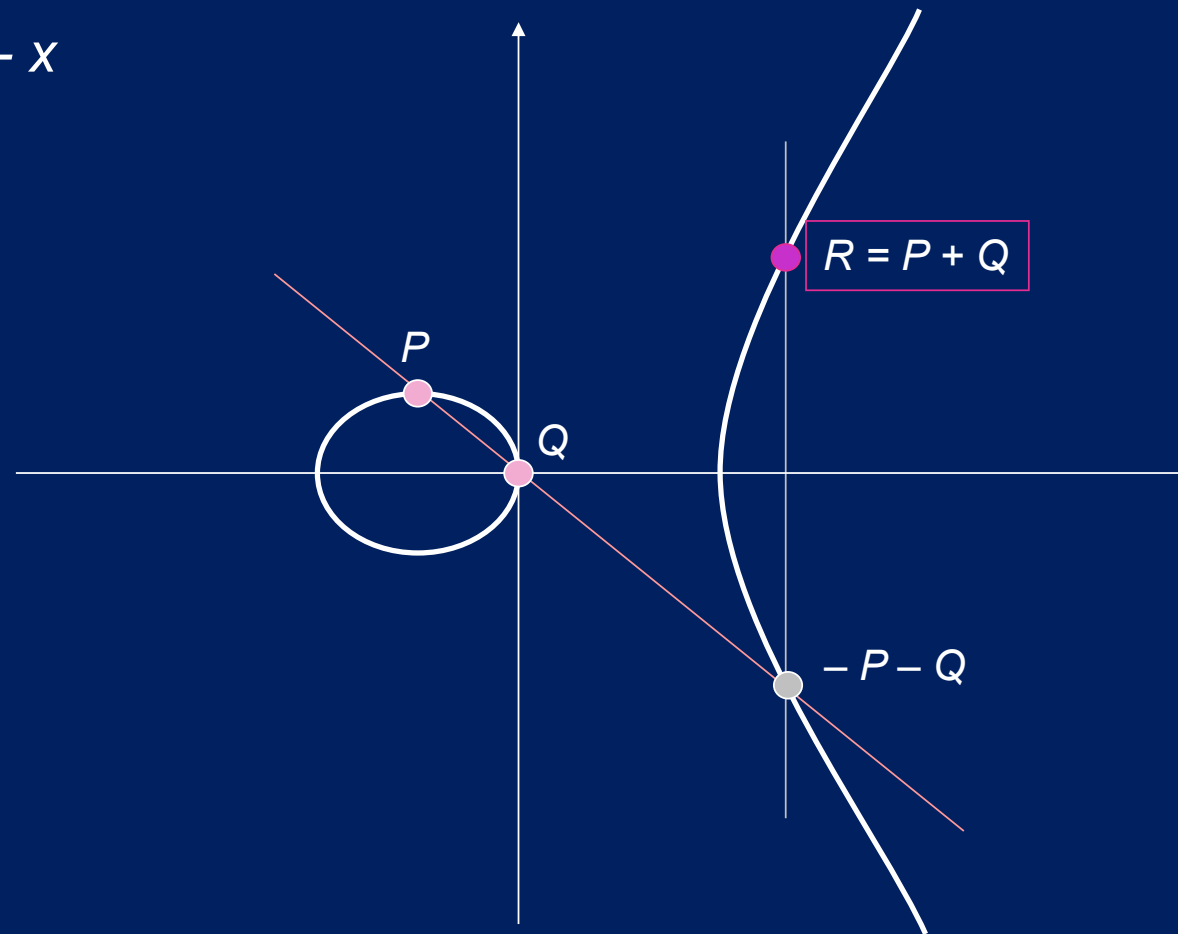
Elliptic Curve Cryptography

Elliptic curve cryptography

- ECC (Elliptic Curve Cryptosystem)
 - instead of using modular arithmetic, operations are executed on a 2D (elliptic) curve
- problems as complex as the ones on modular arithmetic
 - every operation attackers have to do is more computationally demanding
 - use shorter keys ($\sim 1/10$)
 - much faster for legitimate users
- applications
 - digital signatures = ECDSA
 - key agreement = ECDH

Elliptic curve arithmetics

$$y^2 = x^3 - x$$



Key derivation

2. KDF (Key Derivation Function)

- symmetric keys must be random
- ... users prefer passwords (or passphrases)
 - simple and predictable, not really random in the best case
- $K = \text{KDF}(P, S, \text{iterations})$
 - P = password or passphrase
 - S = salt (make K less predictable)
 - iterations = n. of times the same function is executed
 - slow down attackers when performing brute force attacks
 - iterations = 4096 (Kerberos, 2005), 2000 (iOS3), 10000 (iOS4), 5000 (LastPass JS client, 2011), 100000 (LastPass server)
- KDF originally based crypto hashes
 - PBKDF2 (RFC-2898) uses SHA-1, $|S| \geq 64$, itc ≥ 1000
 - HKDF (RFC-5869) uses HMAC

Post-quantum cryptography

DH, RSA and quantum computers

- DH ~ solve the discrete logarithm
 - if $A = g^x \bmod p$ then $x = \log_g A \bmod p$
 - multiplications ~ number of trials multiple of p
 - exponentially grows with the number of bits of p
 - more efficient algorithms exist (e.g. Pohlig–Hellman, number field sieve)
 - all are exponential, though
 - if you can solve the discrete logarithm polynomially, you can also factorise numbers polynomially starting from those results, and vice versa
- however... the Shor's algorithm appeared...

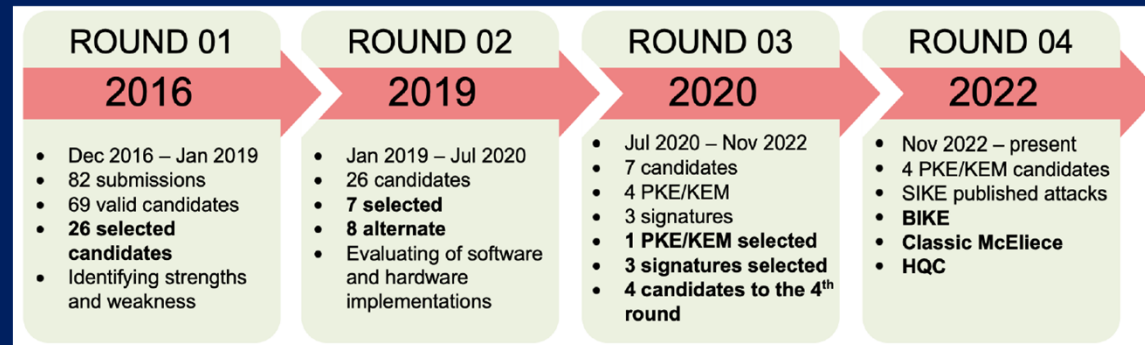
L'algoritmo di Shor

- proposed by Pete Shor to factorize integers
- very fast on ... quantum computers
 - logarithmic complexity
 - $N (\sim O((\log N)^3))$
- still, quantum computers able to execute Shor's algorithm on non-trivial numbers are not available (yet)
 - few bits, a lot of bits needed to perform error correction
 - (2012) a 7 qubit computer solved $21 = 3 \times 7$
 - (2014) adiabatic quantum computation factored $56153 = 233 \times 241$
 - (2023) IBM Osprey, 443 quantum bit
- RSA, DSA, and ECC versions would become weak immediately weak
 - harvest now, decrypt later...

Post-quantum cryptography (PQC)

- urgently needed a new class of difficult problems that are also exponential on quantum computers
- 6 approaches (so far)
 - Lattice-based cryptography
 - Multivariate cryptography
 - Hash-based cryptography
 - Code-based cryptography
 - Isogeny-based cryptography
 - Symmetric key quantum resistance

NIST challenge for PQC



- **August 14th selected...**
 - 3 Finalized Post-Quantum Encryption Standards
 - CRYSTALS-Kyber algorithm → ML-KEM
 - Module-Lattice-Based Key-Encapsulation Mechanism
 - CRYSTALS-Dilithium algorithm --. ML-DSA
 - Module-Lattice-Based Digital Signature Algorithm
 - SpHincs+ algorithm -> SLH-DSA (Backup nel caso trovassero debolezze nei metodi basati su reticoli)
 - Stateless Hash-Based Digital Signature Algorithm
- **new calls ongoing to select new (even better if radically different) families of hard problems**

Quantum computers vs. symmetric cryptography

- symmetric cryptography and hash functions are, as far as we know at the moment, not very affected by quantum computers
- Grover's algorithm
 - doubles the speed of attacks against known algorithms
 - solved by doubling the key length
- hence, no new and completely different technology is needed
 - engineering and incremental updates should work

Public key certificates



How to distribute asymmetric keys

- asymmetric crypto works if...
 - private key is secret!
 - public key is widely distributed
- **problem:** *who checks the link between a key and the owner?*
- **solution #1:** OOB exchange
 - well, it does not work
- **solution #2:** use digital certificates
 - but must understand the certificate format
 - and trust who emits certificates

Certificato a chiave pubblica

“A data structure used to securely bind a public key to some attributes”

- typically it binds a key to an identity
 - other associations are possible too
 - e.g. IP address, email addresses, DNS
- digitally signed by the issuer
 - the Certification Authority (CA)
- limited lifetime
- can be revoked on request, both by the user and the issuer

PKI (Public-Key Infrastructure)

- is the infrastructure ...
- technical and administrative ...
- put in place for the creation, distribution and revocation of public key certificates

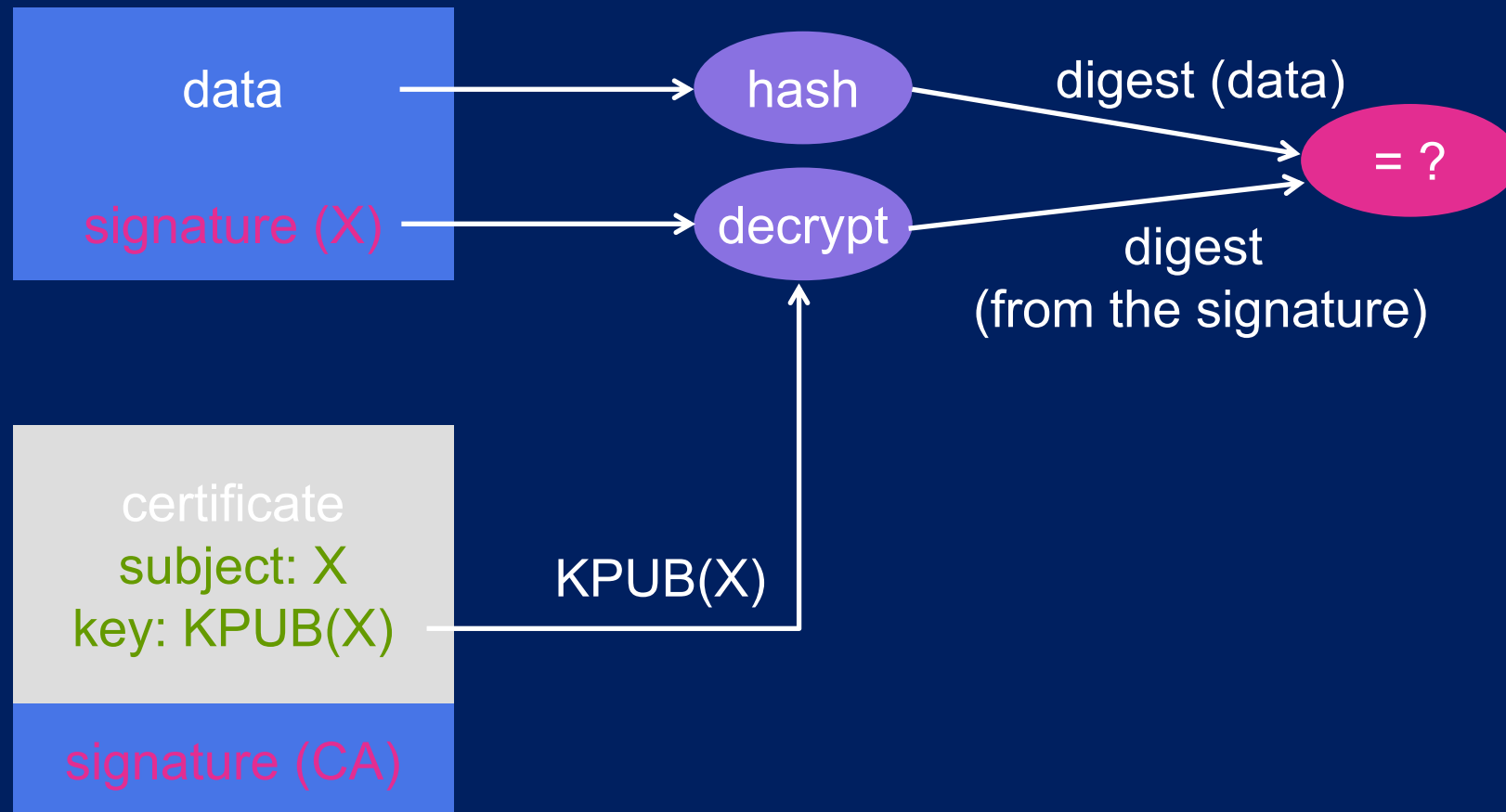
A X.509 certificate

13/04/2026 - Cryptography II - © C. Bas

Certificate

Cataldo Basile	Politecnico di Torino CA	EuroPKI Italian CA	EuroPKI root CA
Subject Name			
Country	IT		
Organization	Politecnico di Torino		
Common Name	Cataldo Basile		
Issuer Name			
Country	IT		
Organization	Politecnico di Torino		
Common Name	Politecnico di Torino CA		
Validity			
Not Before	Mon, 28 Dec 2020 18:17:13 GMT		
Not After	Tue, 31 Dec 2030 23:59:59 GMT		
Subject Alt Names			
Email Address	cataldo.basile@polito.it		
Public Key Info			
Algorithm	RSA		
Key Size	2048		
Exponent	65537		
Modulus	C3:B8:62:3F:02:93:72:E2:89:D1:22:36:E7:57:18:07:45:15:D5:1E:FD:2E:1A:EB:B5:F6:2...		
Miscellaneous			
Serial Number	03:95		
Signature Algorithm	SHA-256 with RSA Encryption		
Version	3		
Download	PEM (cert) PEM (chain)		
Fingerprints			
SHA-256	D0:3C:8E:98:61:93:44:CB:8F:BB:41:81:0E:E6:46:05:12:BC:D0:16:D6:C1:78:C9:AD:52...		
SHA-1	11:2D:63:29:F0:20:68:92:DF:34:68:C7:48:71:A6:0F:09:A0:54:2D		

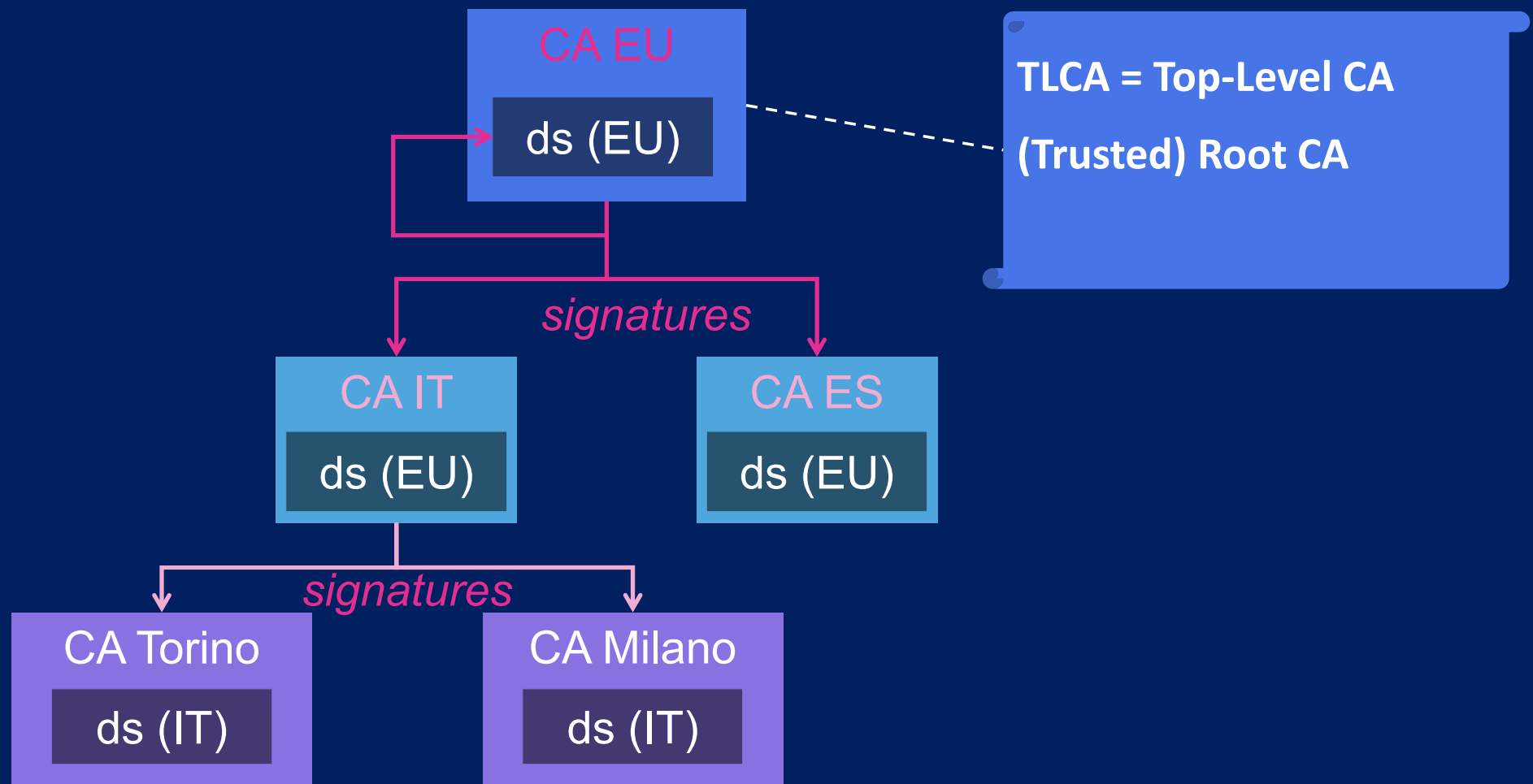
Verification of a signature / certificate



Verifica di una firma / certificato

- how to verify that a public-key certificate (signed by CA1) is authentic?
 - ... the public-key certificate of CA1 is required (which will be signed by CA2)
- How to verify the last one ?
 - ... the public-key certificate of CA2 is required (which will be signed by CA3)
- ... and so on ...
- necessary to have an infrastructure (hierarchical?) for certification and distribution of public-key certificates

Certification hierarchy



Certificate revocation

- any certificate can be revoked before its expiration date:
 - on request by the owner (subject)
 - autonomously by the creator (issuer)
- when a signature is verified, the receiver must check that the certificate was valid at signature time
- check is under the responsibility of the receiver
 - relying party, RP

Meccanismi di revoca

- CRL (Certificate Revocation List)
 - list of revoked certificates
 - signed by the CA or by a delegated party
- OCSP (On-line Certificate Status Protocol)
 - a server responds with information about the certificate status
 - responses must be signed by the server

Electronic documents

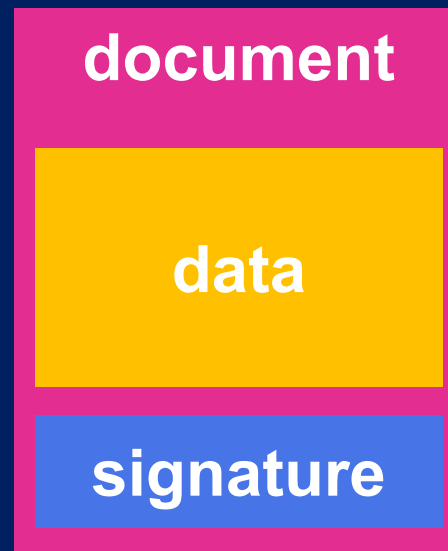
Secure electronic documents

- **electronic document** = any representation of documents with electronic means
- **digital document** = digital representation
- **requires the same properties as the physical ones**
 - integrity and authentication of the originator
 - impossible to forge fake documents
 - usability
 - the format must allow reading the content and accessing the signature
 - long-term storage
 - read and verify also years or decades later

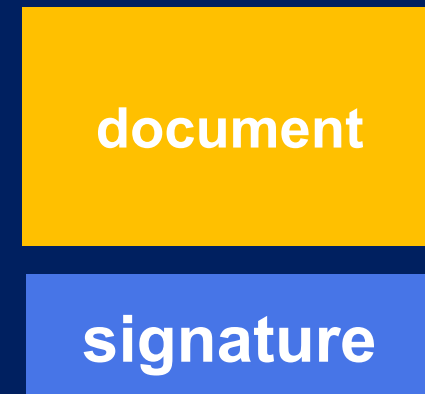
Signed documents format



*enveloping
signature
(es. PKCS#7)*

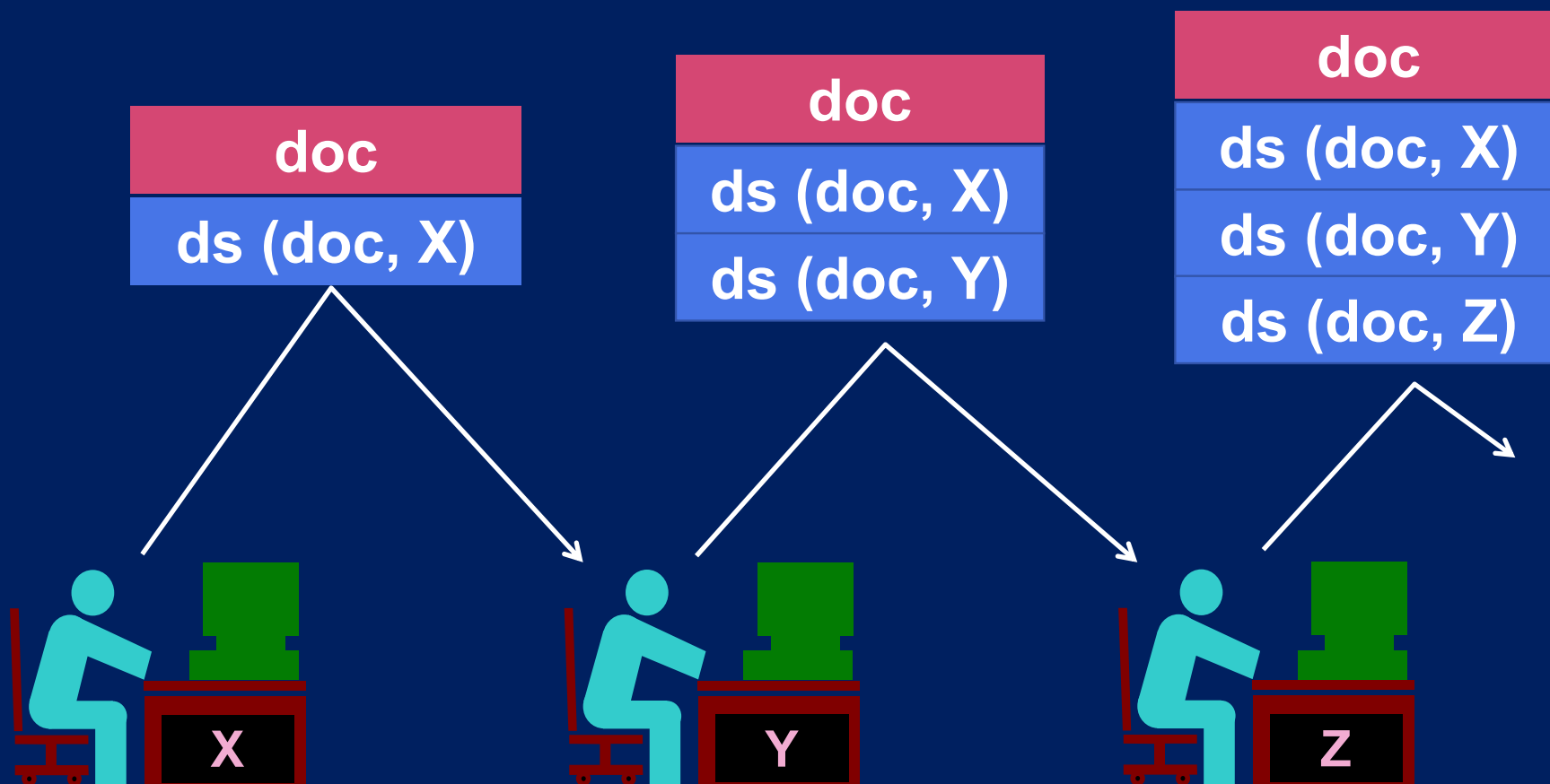


*enveloped
signature
(es. PDF)*

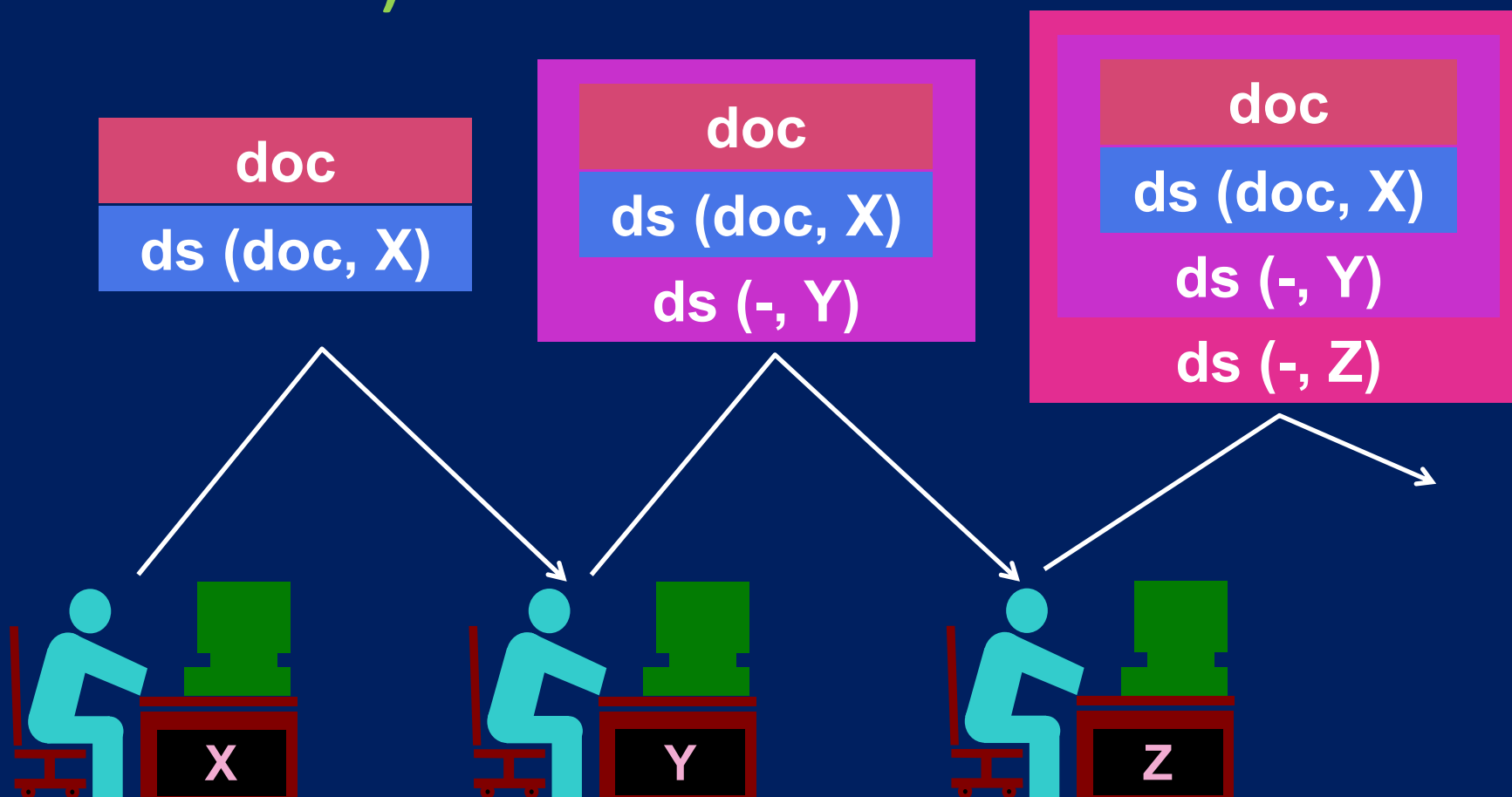


*detached
signature
(es. PKCS#7)*

Multiple signatures (parallel / independent)

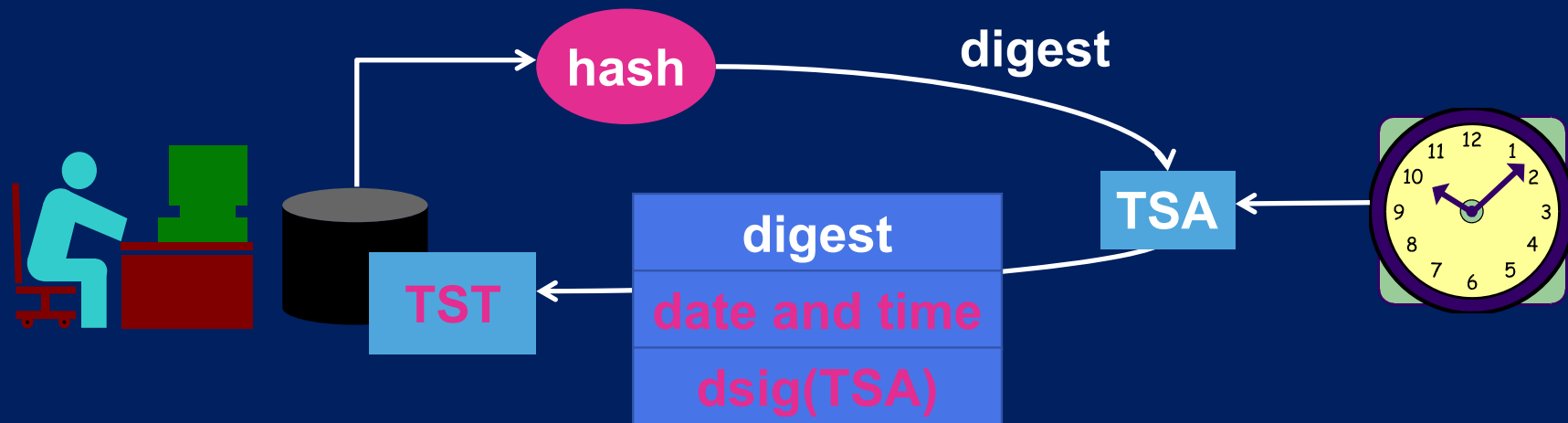


Multiple signatures (sequential / hierarchical)



Time-stamping

- evidence that some data have been created before a certain date
- TSA (Time-Stamping Authority)
- RFC-3161:
 - request protocol (TSP, Time-Stamp Protocol)
 - formati (TST, Time-Stamp Token)



Qualified Electronic Signature (QES)

- QES
 - digital signatures (1) that uses su un Qualified Certificate and (b) generated using ad hoc secure digital signature devices
 - should meet legal requirements
 - as good as hand written pensati per essere analoghi alle firme manuali
 - time stamp da TSA fidate
 - can be used in a court
- keys are from a Qualified Certificate
 - includes attributes that clarify purpose and scope of the key usage
 - e.g., should indicate it is for digital signatures
 - from trusted CAs recognized by the (individual) states

Digital signatures formats

- CMS Advanced Electronic Signatures (CAAdES)
 - ETSI TS 101 733 and 734
- XML Advanced Electronic Signatures (XAdES)
 - ETSI TS 101 903/904 (based on XML-dsig)
- PDF Advanced Electronic Signature Profiles (PAdES) / ISO-32000 format (PDF)
 - ETSI TS 102 778
- firme detached (ASiC = Associated Signature Containers)
 - container (based on the ZIP format) that associates electronic documents with detached signatures (CAAdES or XAdES) and TS
 - ETSI TS 119 162
- more standards are available
 - [CMS] Cryptographic Message Syntax
 - [ESS] Enhanced Security Services